Introduction To Artificial Intelligence

Lecture four

Single agent (vs. multiagent):

- An agent operating by itself in an environment is Single agent (e.g. Crossword Puzzle, Medical Diagnosis, Refinery Controller)
- Multi agent is when other agents are present (e.g. Chess with a clock, Taxi Driving)

PEAS

- Agent: Interactive English tutor
- Performance measure: Maximize student's score on test
- Environment: Set of students
- Actuators: Screen display (exercises, suggestions, corrections
- Sensors: Keyboard

THE STRUCTURE OF AGENTS

How do the insides of agents work In addition their behaviours

A general agent structure

$$agent = architecture + program\ .$$

1- Agent programs Implement the agent function to map percepts (inputs) from the sensors to actions (outputs) of the actuators.

2-Agent architecture The computing device with physical sensors and actuators

E.g., an ordinary PC or a specialized computing device with sensors (camera, microphone, etc.) and actuators (display, speaker, wheels, legs etc.)

Steps done under the agent architecture

1. Sensor's data → Program inputs (Percepts)
2. Program execution
3. Program output → Actuator's actions

Kinds of agent program

– *Table-driven agents -> doesn't work well!*
– Simple reflex agents
– Model-based reflex agents
– Goal-based agents
– Utility-based agents

Table-driven agents

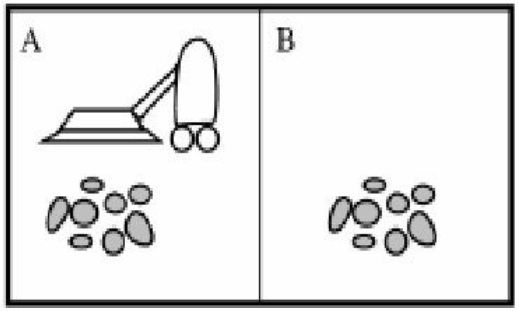
```
function TABLE-DRIVEN-AGENT(percept) returns an action
   persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

   append percept to the end of percepts
   action ← LOOKUP(percepts, table)
   return action
```

–Take the current percept as the input

– The table—an example of which is given for the vacuum world explicitly represent the agent functions that the agent program embodies. To build a rational agent in this way, we as designers must construct a table that contains the appropriate action for every possible percept sequence.

– Agent functions depend on the entire percept sequence

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮



Drawbacks of table driven agent:

(a) no physical agent in this universe will have the space to store the table,

(b) the designer would not have time to create the table,

(c) no agent could ever learn all the right table entries from its experience, and

(d) even if the environment is simple enough to yield a feasible table size, the designer still has no guidance about how to fill in the table entries.

Simple Reflex Agents

- is the simplest kind of agent, these agents select actions based on the current percept, ignoring the rest percept history
  - Memory less
  - Respond directly to percepts

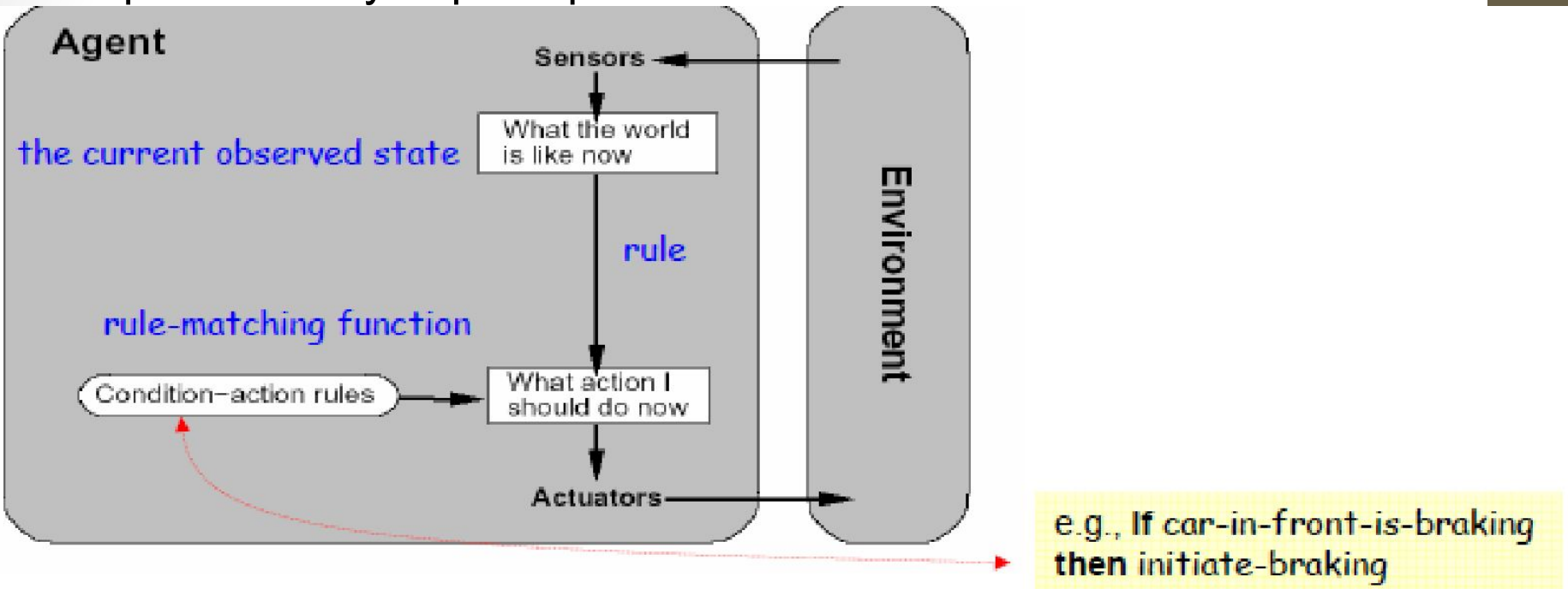


Figure gives the structure of this general program in schematic form, showing how the condition–action rules allow the agent to make the connection from percept to action.

We use rectangles to denote the current internal state of the agent's decision process, and ovals to represent the background information used in the process.

Simple Reflex Agents

Example: the vacuum agent introduced previously

– It's decision is based only on the current location and on whether that contains dirt

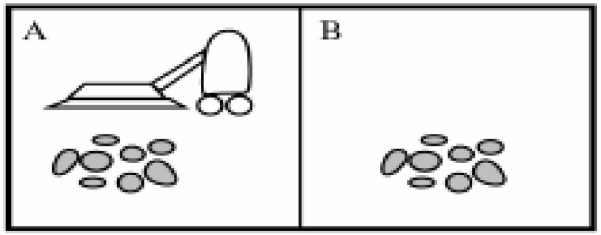
– Only 4 percept possibilities/states (instead of 4^T)

[A, Clean]

[A, Dirty]

[B, Clean]

[B, Dirty]




```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Simple Reflex Agents

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  static: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← RULE-ACTION[rule]
  return action
```

Problems with simple reflex agents

– Work properly if the environment is fully observable

– Couldn't work properly in partially observable environments

– Limited range of applications

-Randomized to escape infinite loops vs. deterministic simple reflex agent

E.g., the vacuum-cleaner is deprived of its location sensor